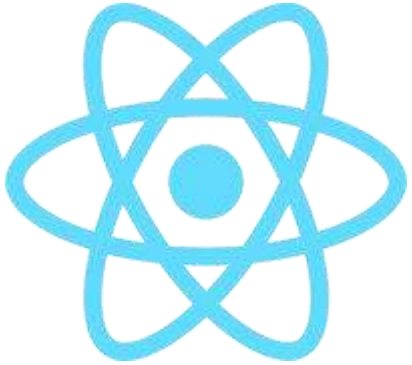
React



Router

Fino a questo momento ci siamo limitati ad avere e gestire una sola pagina. Ma React ci permette di gestire più pagine attraverso la libreria **Router**.

Questa libreria non è presente di default nella libreria React, quindi va importata a livello di progetto e quindi aggiunta nel file **package.json**. Andiamo su google e cerchiamo «react router»

React Router

main

Made by Remix

Getting Started

- Feature Overview
- Tutorial
- Examples
- FAQs
- Main Concepts

Upgrading

- Migrating to RouterProvider
- Upgrading from v5
- Migrating from @reach/router

Routers

- Picking a Router
- createBrowserRouter
- createHashRouter
- createMemoryRouter
- createStaticHandler
- createStaticRouter
- RouterProvider

What's New in 6.4?

v6.4 is our most exciting release yet with new data abstractions for reads, writes, and navigation hooks to easily keep your UI in sync with your data. The new feature overview will catch you up.

I'm New

Start with the tutorial. It will quickly introduce you to the primary features of React Router: from configuring routes, to loading and mutating data, to pending and optimistic UI.

I'm on v5

The migration guide will help you migrate incrementally and keep shipping along the way. Or, do it all in one yolo commit! Either way, we've got you covered to start using the new features right away.

I'm Stuck!

Running into a problem? Chances are you're not the first! Explore common questions about React Router v6.

React

Router

Le route in React si riferiscono alla gestione delle rotte (o percorsi) in un'applicazione web single-page (SPA). In una SPA, invece di caricare nuove pagine dal server, il contenuto viene caricato dinamicamente all'interno della stessa pagina. Questo approccio richiede un sistema per gestire le diverse "viste" o "componenti" che corrispondono a specifici URL, cosa che in un'applicazione React è gestita tipicamente dalla libreria react-router.

Router

react-router è una delle librerie di routing più popolari per React. Essa permette di configurare le route e definire comportamenti in base agli URL, permettendo all'utente di navigare tra diverse parti dell'applicazione e di avere un'esperienza simile a quella di un sito web multi-pagina.

Per installarla dobbiamo eseguire il comando: **npm install react-router-dom**

Router

La versione 6 di `react-router-dom` ha introdotto numerosi cambiamenti e miglioramenti rispetto alla versione 5. Ecco alcuni degli elementi chiave e cambiamenti in React Router v6:

1. **Routes al posto di Switch**: `Switch` è stato sostituito con `Routes`. Questo nuovo componente serve a contenere tutti i vari `Route` e gestisce la selezione di quale componente renderizzare basandosi sull'URL corrente.

2. **Element al posto di Component**: Invece di usare `component` o `render` per specificare il componente da renderizzare, ora si utilizza `element` passando l'elemento JSX direttamente.

Router

3. **Nested Routes**: Le route annidate ora rispecchiano la struttura dei componenti. Le route figlie vengono renderizzate all'interno del componente padre usando `Outlet`.

```
<Route path="users" element={<Users />}>
    <Route path=":id" element={<UserProfile />} />
</Route>
```

4. **useRoutes Hook**: C'è la possibilità di definire le route in maniera imperativa all'interno dei componenti usando l'hook `useRoutes`. Questo può rendere più dinamica la configurazione delle rotte.

```
const routes = useRoutes([
    { path: '/', element: <Home /> },
    { path: 'about', element: <About /> },
    // altri percorsi...
  ]);
```

Router

5. **Relative Links**: I link possono ora essere relativi, il che semplifica la creazione di link all'interno di un set di route annidate.

6. **Automatic Route Ranking**: La versione 6 introduce un sistema di ranking automatico per le route. Non è più necessario preoccuparsi dell'ordine in cui le route vengono dichiarate; il router sceglierà automaticamente la migliore corrispondenza basata sull'URL corrente.

7. **Protected Routes and Layout Routes**: La creazione di route protette e layout routes è ora più intuitiva e può essere gestita usando le route annidate e la logica di autenticazione all'interno dei componenti route.

8. **Miglioramenti nelle prestazioni**: React Router v6 è stato ottimizzato per essere più piccolo e più veloce rispetto alle versioni precedenti.

Gli elementi della libreria router-dom che esamineremo sono : BrowserRouter, Routes, Route.
BrowserRouter è il componente in grado di gestire il Routing della nostra applicazione, mentre **Route** ci permette di creare e gestire uno specifico indirizzo di destinazione.
Le Route devono essere wrappate dal component **Routes**

Di seguito la sintassi, abbiamo anche creato 3 componenti per simulare tre differenti pagine di un sito web

BrowserRouter

```
import { Routes, Route, BrowserRouter } from 'react-router-dom';
import Home from './pagine-route/Home';
import About from './pagine-route/About';
import Profile from './pagine-route/Profile';
const ProvaRotte = () => {
  return (
    <BrowserRouter>
    <Routes>
        <Route path="/" element={<Home />} />
        <Route path="about" element={<About />} />
        <Route path="profile" element={<Profile />} />
    </Routes>
</BrowserRouter>
  )
}

export default ProvaRotte
```

Per navigare all'interno dell'applicazione, utilizziamo un altro componente chiamato Link, che importiamo da react-router-dom. Creiamo rapidamente una barra di navigazione (nav) all'interno del nostro <Router> ma al di fuori dei <Routes>.

```
const ProvaRotte = () => {
  return (
    <BrowserRouter>
     <nav className='d-flex flex-row gap-2 justify-content-center'>
        <Link to='/'>Home</Link>
        <Link to='/about'>About</Link>
        <Link to='/profile'>Profilo</Link>
      </nav>
    <Routes>
        <Route path="/" element={<Home />} />
        <Route path="about" element={<About />} />
        <Route path="profile" element={<Profile />} />
    </Routes>
</BrowserRouter>
```

Per fare una prima prova possiamo aprire ora il nostro browser e nella url principale ci aggiungiamo /chisono cosi come configurato nel BrowserRouter :

All'interno della barra di navigazione, ci sono componenti `<Link>`, che definiscono i percorsi all'interno dell'applicazione. Per le voci di menu come "Home", "About" e "Profile", utilizzeremo tag come `<Link to="/about">About</Link>` e `<Link to="/profile">Profile</Link>`.

Abbiamo ottenuto una barra di navigazione pratica che ci consente di spostarci tra le varie sezioni dell'app senza necessità di ricaricare la pagina .

Il contrasto tra l'uso del componente `<Link>` e un ancor tag tradizionale come `<a href="/about">About</a>` è notevole. Mentre `<a href>` provoca il ricaricamento dell'intera applicazione, `<Link>` consente una transizione istantanea tra le viste, migliorando sensibilmente l'esperienza utente. Questo approccio è vitale in una Single Page Application (SPA), poiché si intende evitare il completo ricaricamento dell'app al cambio di vista, preservando lo stato dell'applicazione e garantendo una navigazione fluida.

In caso di navigazione verso un percorso non definito, come "/profile/1", l'utente vedrà solamente la barra di navigazione, che è progettata per essere sempre visibile. Per gestire queste situazioni, implementiamo un componente `ErrorPage` che verrà mostrato qualora l'utente acceda a un indirizzo inesistente nell'app.

Con l'introduzione di React Router v6, non è più necessario collocare la route di `ErrorPage` come ultima nell'elenco delle routes. Grazie ad un sistema di matching dei percorsi migliorato, l'algoritmo esamina prima le route definite e, in assenza di corrispondenze, renderizza `ErrorPage`, evitando così la visualizzazione di contenuti non previsti.

```
<Routes>
    <Route path="/" element={<Home />} />
    <Route path="/about" element={<About />} />
    <Route path="*" element={<ErrorPage></ErrorPage>}/>
    <Route path="/profile" element={<Profile />} />
```

Template e useParams

All'interno di un'applicazione, è comune l'esigenza di utilizzare dei template. Ad esempio, si potrebbe avere una pagina generica di profili e diverse pagine specifiche per ciascun profilo utente, come "profilo/1234", dove "1234" è l'ID dell'utente.
Invece di creare una route per ogni singolo profilo, che sarebbe insostenibile, usiamo dei template di route. Un template di route è un componente che viene utilizzato in base a un certo pattern di percorso.
Creiamo quindi un componente che chiameremo SingleProfile, il quale sarà usato per visualizzare le informazioni specifiche di un utente basandoci sul suo ID. Nella nostra App.js, definiamo una nuova route come profiles/:id, dove :id è un parametro che cattura l'ID dall'URL.

```
const ProvaRotte = () => {
  return (
    <BrowserRouter>
     <nav className='d-flex flex-row gap-2 justify-content-center'>
        <Link to='/'>Home</Link>
        <Link to='/about'>About</Link>
        <Link to='/profile'>Profilo</Link>
        <Link to='/profile/1'>Single profile</Link>
      </nav>
    <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<ErrorPage></ErrorPage>}/>
        <Route path="/profile" element={<Profile />} />
        <Route path="/profile/:id" element={<SingleProfile />} />
    </Routes>
</BrowserRouter>
  )
```

Ora sicuramente avrò necessità di accedere a l'id che viene passato nella Route e per fare questo abbiamo un nuovo hook lo useParam che restituisce un oggetto con i valori dei parametri.

```
const SingleProfile = () => {
  const params = useParams();
  console.log(params);
  return <div>{`Questo è il  Single Profile con id ${params.id}`}</div>;
};

export default SingleProfile;
```

useNavigate

React Router ci offre anche la possibilità di reindirizzare gli utenti programmaticamente. In precedenza, si usava useHistory, ma nelle versioni più recenti si usa useNavigate. Questo ci fornisce una funzione che possiamo invocare per reindirizzare l'utente. Possiamo usarla in ErrorPage

```
const ErrorPage = () => {
  const navigate = useNavigate();
  return (
    <div>
      Pagina non trovata<br></br>
      <button className='btn btn-success' onClick={() => navigate("/")}>
        Back Home
      </button>
    </div>
  );
};

export default ErrorPage;
```

Se volessimo implementare un pulsante che reindirizza l'utente alla pagina precedente, potremmo passare un numero negativo a navigate, come **navigate(-1),** che equivale a cliccare il pulsante "Indietro" del browser.

Rotte annidate

In pratica stiamo già gestendo rotte annidate con l’esempio sui profile.
Per gestire in modo più efficace questa tipologia di situazione, possiamo inserire delle route una dentro l'altra, quindi andiamo a richiamare la route in questo modo come componente e passiamo come "children" le route che vogliamo inserire.

```
<Route path="/profile" element={<Profile />} >
  <Route path="/profile/:id" element={<SingleProfile />} />
  <Route path="/profile/me" element={<SingleProfile />} />
</Route>
```

Cosa cambia effettivamente? Al momento, il comportamento sembra errato perché, accedendo a "profilo/1", otteniamo "profili" anziché "profili/ID". Questo succede perché è necessario configurare la route "profilo" per farle riconoscere che è una parent route con route annidate al suo interno.

Rotte annidate

```
const Profile = () => {
  return (
    <div>
      <h1>Profile Page</h1>
      <nav className='d-flex flex-row gap-2 justify-content-center'>
        <Link to='me'>Il mio profilo</Link>
        <Link to='/profile/2'>Profilo 2</Link>
      </nav>
      <div>
        <Outlet />
      </div>
    </div>
  );
};
```

Come vediamo per fargli capire che è una rotta padre nel codice del componente andiamo ad aggiungere il componente Outlet

```
const ProvaRotte = () => {
  return (
    <BrowserRouter>
     <nav className='d-flex flex-row gap-2 justify-content-center'>
        <Link to='/'>Home</Link>
        <Link to='/about'>About</Link>
        <Link to='/profile'>Profilo</Link>
        <Link to='/profile/1'>Single profile</Link>
      </nav>
    <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="*" element={<ErrorPage></ErrorPage>}/>
        <Route path="/profile" element={<Profile />} >
          <Route path="/profile/:id" element={<SingleProfile />} />
          <Route path="/profile/me" element={<MyProfile />} />
        </Route>
    </Routes>
</BrowserRouter>
```

useRoutes

Andiamo a creare un file routes.js e importiamo tutti gli import dei routes e creiamo un array di oggetti che riprenda la struttura dei routes.

```
import React from 'react'
import Home from './pagine-route/Home';
import About from './pagine-route/About';
import Profile from './pagine-route/Profile';
import ErrorPage from './pagine-route/ErrorPage';
import SingleProfile from './pagine-route/SingleProfile';
import MyProfile from './pagine-route/MyProfile';
```

```
export const routes = [
    {
      path: "/",
      element: <Home />,
    },
    {
        path: "/about",
        element: <About />,
      },
      {
        path: "/profile",
        element: <Profile />,
        children: [
          {
            path: "me",
            element: <MyProfile />,
          },
          {
            path: ":id",
            element: <SingleProfile />,
          },
        ],
      },
      {
        path: "*",
        element: <ErrorPage />,
      },
]
```

useRoutes

Ora dal componente dove avevamo i Link e le Routes togliamo tutto e importiamo lo hook useRoutes e il file routes.js

```
import React from 'react'
import { useRoutes } from 'react-router-dom';
import { routes } from './routes';

const ProvaRotte = () => {
  let element=useRoutes(routes);
  return element;
}

export default ProvaRotte
```

useRoutes

Ora andiamo dento App.js e creiamo la nostra struttura per i Link.

```
return(
  <BrowserRouter>
   <nav className='d-flex flex-row gap-2 justify-content-center'>
      <Link to='/'>Home</Link>
      <Link to='/about'>About</Link>
      <Link to='/profile'>Profilo</Link>
      <Link to='/profile/1'>Single profile</Link>
    </nav>
  <ProvaRotte></ProvaRotte>
  </BrowserRouter>
```